

Report

Predictive Maintenance with RAG: Technical Report

Links Dataset: [UCI AI4I 2020 Predictive Maintenance](#) Code: [SanjaiPG/RAG-Maintenance](#)

1. Overview

The system takes a row of sensor readings and a maintenance question, estimates the probability that the machine will fail, explains that estimate, and retrieves passages from public maintenance literature to support a written recommendation. An XGBoost classifier produces $P(\text{failure})$. TreeSHAP ranks the features behind it. Those features, plus a few rule-based condition screens, form a query against a TF-IDF index of 454 passages. The answer is built only from what was retrieved and carries inline citations.

2. Data and leakage control

The dataset has 10,000 records and 339 failures (3.39%). It was split before any modeling into train (8,000 rows, 271 failures), validation (1,000 rows, 34 failures) and test (1,000 rows, 34 failures). The splits are stratified on `Machine failure` with seed 42, and the test set was untouched until the final evaluation.

The model sees six inputs: `Type` (L/M/H), air temperature, process temperature, rotational speed, torque and tool wear. Everything else was dropped:

- `UDI` is a row index with no physical meaning, and trees could memorize ranges of it.
- `Product ID` is a quality prefix plus a serial number. The prefix is already in `Type`, and the serial reflects manufacturing order.
- `Machine failure` is the target.
- `TWF`, `HDF`, `PWF`, `OSF` and `RNF` are the failure mode flags that define the target. In a plant the mode is only known after diagnosis, so using them as inputs would leak the outcome.

3. Models and results

Predicting "no failure" for every row scores 96.6% accuracy, so I used precision, recall, F1, ROC-AUC and PR-AUC. The baseline is logistic regression with balanced class weights, standard scaling and one-hot `Type`. The main model is XGBoost with `scale_pos_weight` of about 28.5, 300 trees, depth 5, learning rate 0.05 and subsampling of 0.8.

Validation comparison (threshold 0.50)

Model	Precision	Recall	F1	ROC-AUC	PR-AUC
Logistic regression	0.1613	0.8824	0.2727	0.9098	0.3586
XGBoost	0.6667	0.7059	0.6857	0.9732	0.7689

Logistic regression catches more failures but raises over five false alarms per real one. XGBoost more than doubles PR-AUC, which fits failures that depend on interactions such as power (torque times speed) and the process-to-air temperature gap.

Threshold selection

Threshold	0.10	0.20	0.30	0.40	0.50
Precision	0.3333	0.4375	0.5098	0.5952	0.6667
Recall	0.8824	0.8235	0.7647	0.7353	0.7059
F1	0.4839	0.5714	0.6118	0.6579	0.6857

F1 was highest at 0.50, so that became the operating point. Below 0.30 the false alarm rate would likely wear out a maintenance crew.

Test set (threshold 0.50, 1,000 rows)

Precision 0.6591, recall 0.8529, F1 0.7436, ROC-AUC 0.9705, PR-AUC 0.8786.

	Predicted healthy	Predicted failure	Total
Actual healthy	951	15	966
Actual failure	5	29	34

4. Explainability and retrieval

Every row goes through `shap.TreeExplainer`, which gives each feature a signed contribution in log-odds. The one-hot `Type` columns are summed back into one feature, and features are ranked by absolute contribution.

The knowledge base has 7 public sources: three NIST pages on condition monitoring, machinery maintenance and diagnostics, NASA NPR 8831.2F, two Fluke articles on vibration monitoring and thermal imaging, and an SKF page on machine health. The text was extracted, cleaned and cut into 180-word passages with a 40-word overlap (454 passages), then indexed

with `TfidfVectorizer` (sublinear TF, unigrams and bigrams, English stop words). Results are ranked by cosine similarity, with at most 2 passages per source.

The query combines the sensor values, the top SHAP drivers and condition screens such as a temperature gap under 8.6 K, power outside its normal band, or high tool wear. Two rules govern the answer. It states that $P(\text{failure})$ is a statistical estimate and not a confirmed diagnosis. An inspection action (thermal imaging, vibration trending, lubricant analysis, baseline comparison) is only recommended if a retrieved passage discusses it, and every statement carries a citation such as `[fluke_thermal_1]` .

5. End-to-end evaluation

I ran 20 test cases, 10 real failures and 10 healthy machines. All recommendations were supported by the retrieved passages, and the queries produced 14 distinct sets of passages, so the output changes with the sensor condition.

Case	True	P(fail)	Result	Main SHAP factors	Guidance
01 (row 425)	1	0.970	TP	Air temp +3.64, speed +1.76	Low heat dissipation; thermal imaging and vibration trending
06 (row 77)	1	0.999	TP	Torque +3.58, tool wear +1.61	Overstrain (60.1 Nm, 215 min); replace tool, check vibration
10 (row 717)	1	0.076	FN	Tool wear -1.27, torque -1.24	Missed at 0.50, caught by the watch band

6. Missed failures

Five of the 34 test failures were missed. One cause is the random failure mode (`RNF`): in this synthetic dataset random failures have no lead-up in the sensors, so every SHAP value can stay negative, as in case 10. The other is borderline degradation, for example tool wear near 180 min with torque around 35 Nm, where the model gives a probability between 0.15 and 0.45. To cover both, the system adds a watch band for probabilities above 0.05 and below the threshold, and a machine in that band gets an inspection recommendation even though it is predicted healthy.

7. Future work

1. Replace TF-IDF with dense embeddings (`bge-small-en-v1.5`) and rerank with a cross-encoder (`bge-reranker-large`).
2. Add split conformal prediction for risk intervals at a chosen confidence level, such as 95%.

3. Add a second model that classifies the failure mode once a failure is predicted, and generate the explanation text with a local LLM using grammar-constrained decoding so every citation points to a retrieved passage.

8. Declarations

Data: UCI AI4I 2020 (CC BY 4.0). Documents: public publications from NIST, NASA, Fluke and SKF. Software: Python 3.10+, scikit-learn 1.5, XGBoost 2.1.4, SHAP 0.46.0, pandas, NumPy, joblib, BeautifulSoup4 and PyPDF.